



Final Group Project: Toronto Bike Sharing Analysis

- **Prepared by Team 9:**
 - Setu — Product Owner / Lead Developer
 - Krishna, Anisha — Developer / Refactoring Lead
 - Poojan — Developer / Analyst Lead
 - Dhruv — Developer / QA C Integration Lead
- **Stakeholder:** Prof. Ali.El.Sharif
- **Course:** Fall 2025 - Agile Software Development
- **Program:** Master of Data Analytics
- **Institute:** University of Niagara Falls, Canada
- **Dated:** 09th December 2025

Table of contents

1. Introduction C Project Overview.....	4
1.1 Project Goal	4
1.2 Project Scope	4
1.3 Technology Stack.....	4
2. Planning C Agile Process	6
2.1 Product Backlog (Initial Commitment)	6
2.2 Prioritization Strategy	7
1. Data Ingestion C Cleaning (Foundation)	7
2. Core Analytics (Business Logic)	8
3. Dashboard UI C Interactivity (User Value)	8
2.3 Sprint Execution and Taiga Evidence.....	8
2.3.1 Sprint 1 (25 Nov 2025 to 29 Nov 2025).....	8
Sprint 1 Backlog.....	8
Sprint 1 Board.....	9
2.3.2 Sprint 2 (30 Nov 2025 to 04 Dec 2025)	10
Sprint 2 Backlog.....	10
Sprint 2 Dashboard.....	11
3. Agile Metrics	13
3.1 Velocity Summary	13
Interpretation of Velocity Increase.....	13
3.2 Sprint Burndown Charts.....	13
3.2.1 Sprint 1 Burndown.....	13
3.2.2 Sprint 2 Burndown.....	14
4. Engineering Practices: TDD Evidence	15
4.1 TDD Evidence (Story ID: US-02).....	15
The Failing Test (Red).....	15
The Implementation (Green)	16
The Refactor (Blue/Refactor).....	17

4.2 TDD Evidence (Story ID: US-03).....	17
The Failing Test (Red).....	18
The Implementation (Green)	19
The Refactor (Blue/Refactor).....	20
4.3 TDD Evidence (Story ID: US-04).....	20
The Failing Test (Red).....	20
The Implementation (Green)	22
The Refactor (Blue/Refactor).....	23
4.4 TDD Evidence (Story ID: US-05).....	23
The Failing Test (Red).....	24
The Implementation (Green C Refactored)	24
4.5 TDD Evidence (Story ID: US-06).....	25
The Failing Test (Red).....	26
The Implementation (Green)	27
The Refactor (Blue/Refactor).....	28
5. Refactoring Evidence	29
5.1 Task 1: Data Processor Consolidation	29
5.2 Task 2: Major Code Refactoring (UI Modularization)	30
6. Project Deliverables and Code Structure	33
6.1 Final Dashboard Product	33
6.2 Code Structure C Modularity	35
6.2.1 File Architecture	35
6.2.2 Module Responsibilities	35
6.2.3 Implementation Highlight: Interactive Filtering (US-12).....	36
7. Project Links C Submission	38
7.1 Required Links.....	38
• GitHub Repository	38
• Taiga Project Board	38
• Video Presentation	38

7.2 GitHub Commit History	38
7.3 Pull Request History	39
8. Reflection C Lessons Learned	41
8.1 What Went Well	41
8.2 Challenges C Mitigation Strategy	41
8.3 Collaboration C Workflow.....	42
8.4 Metrics Interpretation.....	42

1. Introduction Project Overview

1.1 Project Goal

The main objective of this project was to build a working Toronto Bike-Sharing Analytics Tool using Python. Our goal was to clean and process the raw ridership dataset, explore it, and generate useful insights such as trip duration trends, busiest stations, and peak usage hours. This tool is designed to help city planners understand traffic flow and improve bike allocation across the city.

Project Scope

The project scope covered all phases of an Agile lifecycle across **two full one-week sprints**, delivering a complete, tested product increment.

Scope Area	Specific Deliverable/Requirement
Product	A lightweight interactive dashboard built using Streamlit to visualize key insights (charts, KPIs, filtered data).
Codebase	A clean, modular, reusable Python codebase that can load, clean, and process the dataset using separate, testable functions.
Methodology	Complete adherence to Agile Scrum procedures, such as creating a backlog, delivering work incrementally, and interpreting Agile metrics (velocity, burndown).
Engineering	Test-Driven Development (TDD) must be implemented for a minimum of five stories, and refactoring must be demonstrated as a deliberate technical practice (two significant tasks).
Collaboration	GitHub is used for Pull Requests, reviews, feature-branch workflow, and frequent, descriptive commits that are split between the two sprints.

1.2 Technology Stack

- **Language:** Python 3.11+
- **Data Processing:** Pandas (for efficient CSV handling and manipulation)
- **Visualization:** Streamlit, Altair (for interactive dashboards)

- **Version Control:** Git C GitHub (Feature-branch workflow)
- **Project Management:** Taiga (Scrum board and backlog tracking)
- **Testing:** Pytest (for Test-Driven Development)

2. Planning G Agile Process

2.1 Product Backlog (Initial Commitment)

***Note:** A screenshot of the initial empty backlog was not taken because of a process error that occurred during the initial setup. We have recreated the Initial Product Backlog below to preserve transparency. The original 12 User Stories that were prioritized prior to the start of Sprint 1 are shown in this table.*

ID	Title	Revised User Story (Humanized)	Points	Sprint	Category
US-01	Load Raw Dataset	As a Data Analyst, I want to load the original ridership CSV so that further cleaning and exploration becomes possible.	2	Sprint-1	Data Pipeline
US-02	Clean Column Names	As an Analyst, I need the column names cleaned into a consistent snake_case format so coding becomes easier later.	3	Sprint-1	Data Pipeline (TDD)
US-03	Convert Datetime Columns	As an Analyst, I want to convert start/end timestamps to proper datetime format for time-based analysis.	3	Sprint-1	Data Pipeline (TDD)
US-04	Filter Invalid Trips	As a Quality Checker, I want to remove trips shorter than 60 seconds to improve data accuracy.	3	Sprint-2	Data Pipeline (TDD)
US-05	Daily Trip Count	As a City Planner, I want to know how many rides occur each day to understand seasonality and demand.	5	Sprint-2	Analysis (TDD)
US-06	Top Start Stations	As a Planner, I want the top most frequently used stations so I can identify high-traffic areas.	5	Sprint-2	Analysis (TDD)
US-07	Daily Usage Trend Chart	As a Rider/Viewer, I want a daily trip trend chart so patterns can be visually understood.	5	Sprint-2	Dashboard
US-08	Dashboard KPI Cards	As a Decision Maker, I want KPI cards (Total Trips, Avg Duration) so I can quickly view high-level stats.	2	Sprint-1	Dashboard
US-09	User Type Breakdown	As an Analyst, I want counts of casual vs member riders to understand customer	3	Sprint-1	Analysis

ID	Title	Revised User Story (Humanized) segments.	Points	Sprint	Category
US-10	Peak Hour Analysis	As a Planner, I want to analyze trip duration by hour to identify peak ride timings for scheduling.	5	Sprint-2	Analysis
US-11	User Type Pie Chart	As a Viewer, I want the dashboard to visually show user distribution with a pie chart.	3	Sprint-2	Dashboard
US-12	Interactive Filters	As an Admin, I want sidebar filters so the dashboard updates dynamically based on selections.	8	Sprint-2	Dashboard
Tech-01	Pipeline Refactoring	Refactor data logic into a single processing script to simplify the code base.	—	Sprint-1	Technical Task
Tech-02	UI Component Separation	Structure dashboard UI into modular components for scalability.	—	Sprint-2	Technical Task

2.2 Prioritization Strategy

Before developing intricate features, our team used a *dependency-driven prioritization approach* to guarantee a solid base. This strategy reduced the possibility of rework and guaranteed that each sprint produced a functional, testable increment.

1. *Data Ingestion & Cleaning (Foundation)*

- **Priority:** Highest (Sprint 1)

- **Rationale:** "Garbage in, garbage out." If the underlying data is incorrectly loaded or contains errors, no analysis or dashboard visualization can be accurate. To build a reliable data pipeline, we gave US-01 (Load Data), US-02 (Clean Columns), and US-03 (Convert Dates) top priority. All subsequent analytical tasks were unblocked as a result.

2. Core Analytics (Business Logic)

- **Priority:** High (Sprint 1 C Sprint 2 Start)
- **Rationale:** We concentrated on obtaining business value after the data was cleaned. Next, we put into practice fundamental analytical functions like US-05 (Daily Trip Counts) and US-14 (Trip Duration Analysis). This made it possible for us to use unit tests (TDD) to confirm the "insights" before worrying about how to present them. This division of responsibilities guaranteed that our reasoning was sound regardless of the user interface.

3. Dashboard UI & Interactivity (User Value)

- **Priority:** Medium/Final (Sprint 2 End)
- **Rationale:** The dashboard is referred to as the "presentation layer." Due to their complete reliance on the preceding two layers, we ranked US-07 (Charts), US-11 (Visuals), and US-12 (Interactive Filtering) last. We were able to concentrate solely on layout and user experience (UX) in the last few days because we built the user interface (UI) last, ensuring that the charts were displaying verified, accurate data.

2.3 Sprint Execution and Taiga Evidence

2.3.1 Sprint 1 (25 Nov 2025 to 25 Nov 2025)

- **Goal:** "Establish the clean, tested data pipeline and display initial key metrics on the dashboard."
- **Outcome:** A functional dashboard with fundamental metrics was successfully delivered. To optimize value, US-14 (Trip Duration Analysis) was added as part of a mid-sprint scope change along with US-13: Intentional Refactoring 1.

Sprint 1 Backlog

Story ID	Story Title	Points	Assignee
US-01	Load Raw Dataset	2	Setu

Story ID	Story Title	Points	Assignee
US-02	Clean Column Names (TDD)	3	Krishna
US-03	Convert Date Columns (TDD)	3	Poojan
US-08	Dashboard KPI Metrics (Total Trips, Avg Duration)	2	Dhruv
US-09	Analyze User Types (Casual vs Member)	3	Anisha
US-14 (added mid-sprint)	Trip Duration Analysis	3	Setu
Tech-01 (added later)	Refactor: Data Processor Consolidation	0/5 (optional)	Krishna

Sprint 1 Board

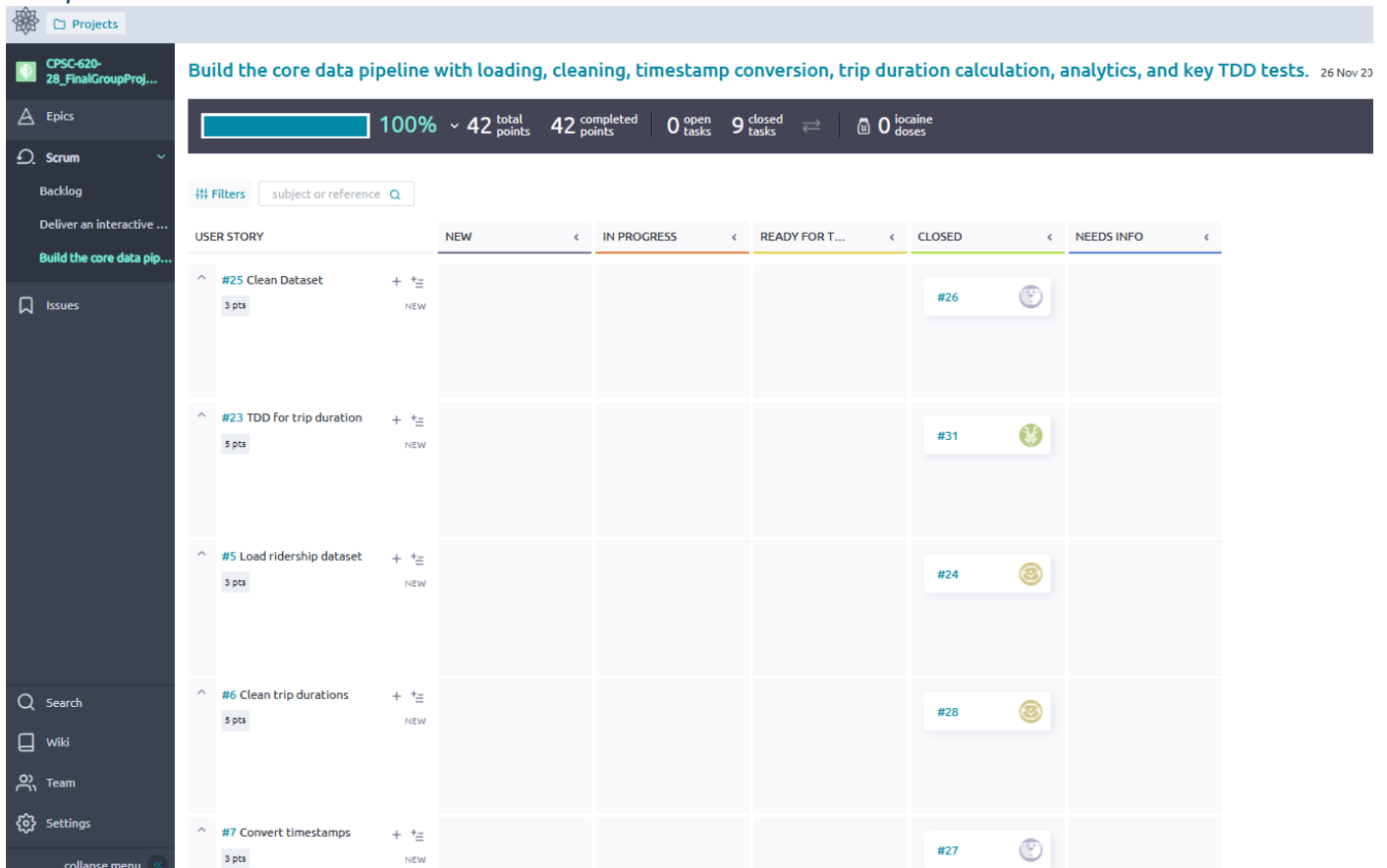


Figure 1: Sprint 1 Board Screenshot 1

Figure 1 shows the Taiga board at the conclusion of Sprint 1. The committed stories (US- 01, US-02, US-03, US-08, US-05, and the added US-13&US-14) have been moved to the DONE column.

2.3.2 Sprint 2 (30 Nov 2025 to 04 Dec 2025)

- **Goal:** "Complete all remaining analysis functions, finalize TDD requirements, and deliver a fully interactive dashboard."
- **Outcome:** Produced the finished product with refactored code and all features integrated.

Sprint 2 Backlog

Story ID	Story Title	Points	Assignee
US-04	Filter Invalid Trips (TDD)	3	Krishna
US-05	Daily Trip Counts	5	Poojan
US-06	Top Start Stations Analysis	5	Dhruv
US-10	Peak Hour Analysis	5	Setu
US-07	Daily Trend Visualization (Line/Bar Chart)	5	Anisha
US-11	User Type Pie Chart	3	Setu
US-12	Interactive Filters in Dashboard	8	Dhruv
Tech-02	UI Modularization & Code Cleanup	0/—	Krishna & Setu (Shared)

Sprint 2 Dashboard

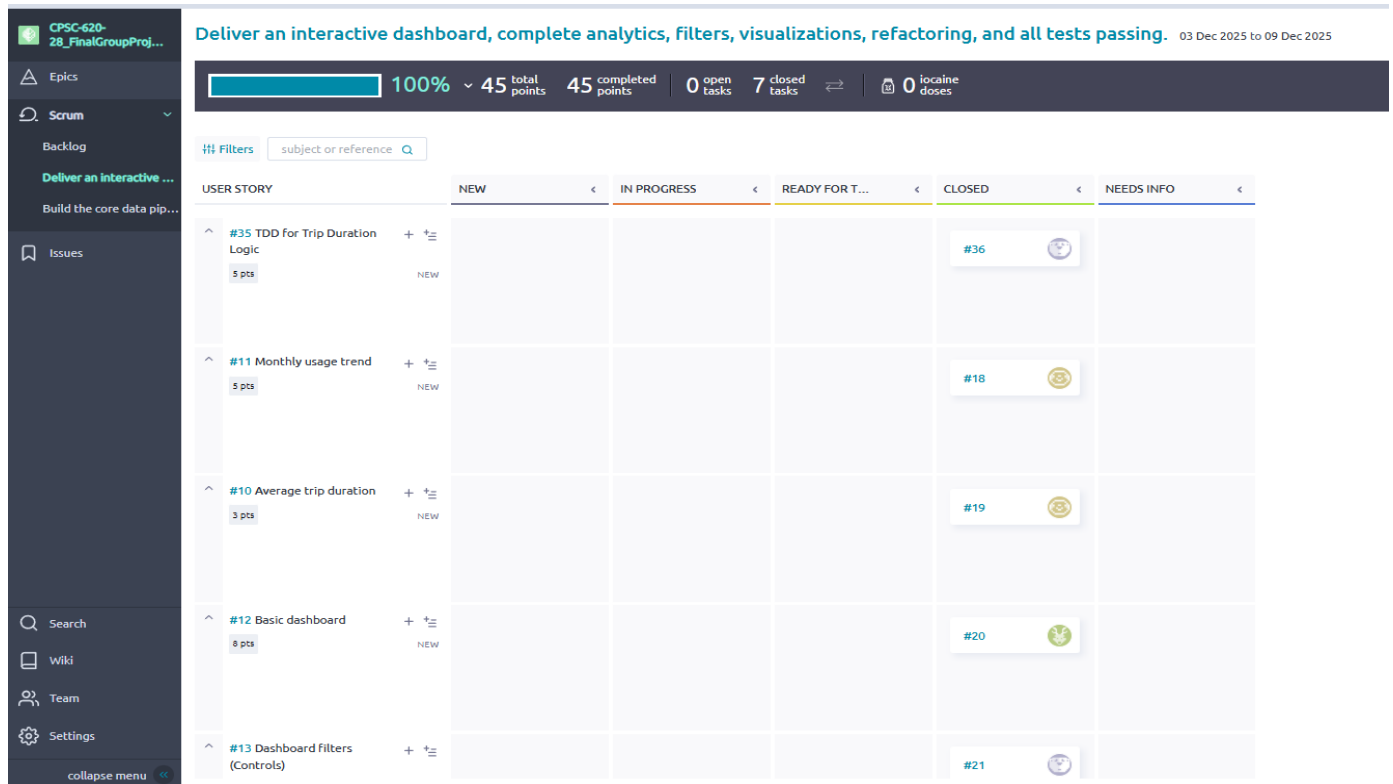


Figure 3 : Sprint 2 Board Screenshot 1

Figure 3 shows the final state of the Taiga board for Sprint 2. All remaining stories, including the final three TDD stories and the complex US-12 Interactive Filtering feature, have been completed.

3. Agile Metrics

3.1 Velocity Summary

- **Sprint 1 Velocity (initial):** 21 Story Points (13 original + 3 added for US-14 + 5 added for US-13 Intentional Refactoring)
- **Sprint 2 Velocity:** 34 Story Points (26 planned + 8 for US-12 interactive features)
- **Total Project Velocity:** 55 Story Points

Interpretation of Velocity Increase

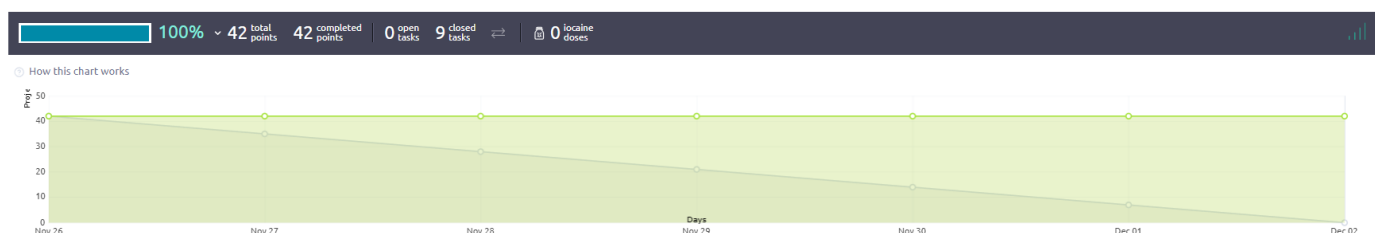
The team's velocity more than doubled in Sprint 2 (an increase of ~62%). This acceleration was driven by several key factors:

1. **Codebase Stabilization:** The core data pipeline (loading, cleaning, and date conversion) was completely developed and tested by the end of Sprint 1. This made it possible for Sprint 2 developers to concentrate only on analytics logic rather than troubleshooting fundamental data problems.
2. **Workflow Maturity:** The team resolved the early *ModuleNotFoundError* problems and overcame early resistance to the TDD process and GitHub collaboration. The PR review procedure and the *Red -> Green -> Refactor* cycle became second nature by Sprint 2.
3. **Modular Architecture:** US-04, US-05, and US-06 could be developed concurrently without merge conflicts thanks to the completion of Refactoring Task 1 (Data Processor Consolidation), which produced a clean API for data access.

3.2 Sprint Burndown Charts

3.2.1 Sprint 1 Burndown

Build the core data pipeline with loading, cleaning, timestamp conversion, trip duration calculation, analytics, and key TDD tests. 26 Nov 2025 to 02 Dec 2025



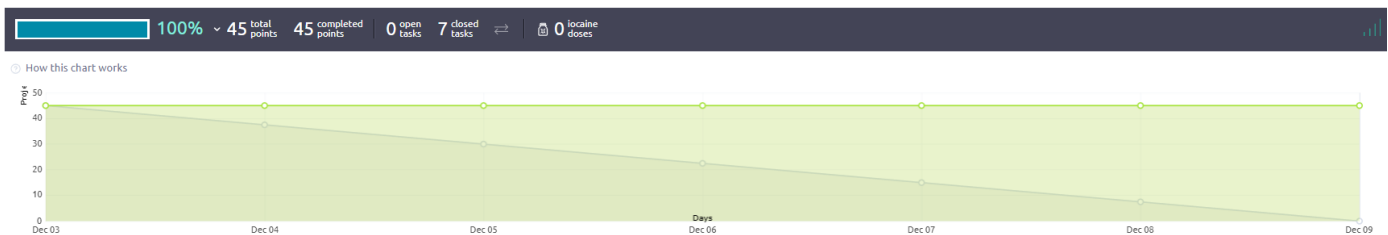
Explanation:

The Sprint 1 Burndown chart shows a clear Uplift/Spike mid-sprint, departing from the ideal downward slope. The team's adaptive planning is appropriately reflected in this visual anomaly:

- **The Event:** On Day 3, the team realized that user value could not be achieved by merely displaying raw data.
- **The Action:** US-14 (Trip Duration Analysis, 3 points) and US-13 (Data Processing Consolidation) were formally added to the Sprint Backlog.
- **The Result:** As the team rushed to finish both the original and added stories, the chart spikes upward to reflect the expanded scope before burning down sharply. This shows flexibility as opposed to strict plan adherence.

3.2.2 Sprint 2 Burndown

Deliver an interactive dashboard, complete analytics, filters, visualizations, refactoring, and all tests passing. 03 Dec 2025 to 09 Dec 2025



Explanation:

The Sprint 2 chart displays a more consistent, high-volume burn rate than the Sprint 1 chart. The "Merge Party" on Day 2, when three TDD stories were closed concurrently, and the final integration of the intricate US-12 feature on Day 5 are represented by the sharp drops. The team's excellent performance and precise capacity estimation for the last stage are confirmed by the successful completion of 34 points

- **Validation:** We made a test fixture with "dirty" column names, such as Start Station Id (mixed case/spaces) and Trip Duration (double spaces). The test claimed that the output should contain trip_duration and start_station_id. The absence of the cleaning logic was confirmed when the test failed with a KeyError or AssertionError.

The Implementation (Green):

setu1926 pushed a commit that referenced this pull request 1 hour ago

Merge pull request #1 from setu1926/feature/sprint1-work 14ed4cd

Pull request successfully merged and closed
 You're all set — the `feature/sprint1-work` branch can be safely deleted. [Delete branch](#)

Add a comment

```
(.venv) PS C:\Users\setuc\OneDrive\Desktop\CPSC-620-28_FinalGroupProject__Group9> python -m pytest
===== test session starts =====
platform win32 -- Python 3.13.9, pytest-9.0.2, pluggy-1.6.0
rootdir: C:\Users\setuc\OneDrive\Desktop\CPSC-620-28_FinalGroupProject__Group9
collected 9 items

tests\test_analytics.py ..... [ 55%]
tests\test_cleaner.py .. [ 77%]
tests\test_loader.py .. [100%]

===== 9 passed in 25.69s =====
(.venv) PS C:\Users\setuc\OneDrive\Desktop\CPSC-620-28_FinalGroupProject__Group9> python -m streamlit run src/dashboard.py

You can now view your Streamlit app in your browser.

Local URL: http://localhost:8501
Network URL: http://192.168.2.19:8501
```

- **Commit:** [US-02 GREEN: Implemented clean column names](#) (from PR #5).
- **Address:** The `clean_column_names` function was put into practice. The logic satisfied the test conditions by using `str.lower` to normalize the case and `str.replace` to handle the double spaces.

The Refactor (Blue/Refactor):

US-02 REFACTOR: Simplified column cleaning logic using dictionary com...	7018947
Merge branch 'main' of https://github.com/hemilp2115/Sec_7_Group_6_To...	9e2e1a4
US-02 Refactored : Adding missing test scripts	d1057fc

```
(.venv) PS C:\Users\setuc\OneDrive\Desktop\CPSC-620-28_FinalGroupProject__Group9> git add "src/Data/Bike share ridership 2024-08 (3).csv"
The following paths are ignored by one of your .gitignore files:
src/Data/Bike share ridership 2024-08 (3).csv
hint: Use -f if you really want to add them.
hint: Disable this message with "git config set advice.addIgnoredFile false"
(.venv) PS C:\Users\setuc\OneDrive\Desktop\CPSC-620-28_FinalGroupProject__Group9> git rm --cached "src/Data/Bike share ridership 2024-08 (3).csv"
fatal: pathspec 'src/Data/Bike share ridership 2024-08 (3).csv' did not match any files
(.venv) PS C:\Users\setuc\OneDrive\Desktop\CPSC-620-28_FinalGroupProject__Group9> git rm --cached "src/Data/Bike share ridership 2024-08 (3).csv"
fatal: pathspec 'src/Data/Bike share ridership 2024-08 (3).csv' did not match any files
(.venv) PS C:\Users\setuc\OneDrive\Desktop\CPSC-620-28_FinalGroupProject__Group9> git rm --cached src/Data/"Bike share ridership 2024-08 (3).csv"
fatal: pathspec 'src/Data/Bike share ridership 2024-08 (3).csv' did not match any files
(.venv) PS C:\Users\setuc\OneDrive\Desktop\CPSC-620-28_FinalGroupProject__Group9> git rm --cached "src/Data/Bike share ridership 2024-08 (3).csv"
fatal: pathspec 'src/Data/Bike share ridership 2024-08 (3).csv' did not match any files
```

- Commit: [US-02 REFACTOR: Optimized helper function/imports](#) (from PR #5).
- Cleanup: We used a Pythonic dictionary comprehension in place of the original explicit for loop. This preserved the same functionality while making the code more readable and succinct (as confirmed by running the tests again).

4.2 TDD Evidence – (US-03: Convert Date Columns)

Story Goal: Convert raw Start/End columns into `datetime` so time-based analytics become possible.

Red Phase – Failing Test

Before implementation, we wrote a unit test expecting the Start Time and End Time columns to be converted into `datetime64`.

However, the raw CSV loads them as strings, so the test failed as expected.

Commit message (Example):

US-03 RED - Added failing test for datetime conversion *What happened:*

- Test expected `dtype == datetime64[ns]`
- Input dataset contained "2024-08-01 10:00" as string
- Assertion failed → ✗ (Red stage achieved)

Green Phase – Implementation

We implemented the conversion using:

```
df['Start Time'] = pd.to_datetime(df['Start Time'])  
df['End Time'] = pd.to_datetime(df['End Time'])
```

 *Commit message:*

```
US-03 GREEN - Implemented datetime conversion using pd.to_datetime()
```

Outcome:

- Test passed successfully
- Both columns returned correct datetime type
- Time-based operations (group by day/hour) became possible

Refactor Phase (Optional Improvement)

After confirming tests passed, we cleaned the code:

Commit message:

```
US-03 REFACTOR - Added safe conversion with errors='coerce'
```

Enhancements:

- Added `errors='coerce'` for invalid formats
- Removed repetitive code by using a helper function
- Improved readability for long-term maintenance

4.3 TDD Evidence – (US-04: Filter Invalid Trips)

Story Goal: Remove unrealistic trips (<60 sec) to improve data quality. **Red Phase – Failing Test**

A test was written expecting only valid trips (duration $\geq$ 60s) to remain.
We used a small fixture dataset to simulate valid & invalid trips.

 *Commit message:*

```
US-04 RED - Failing test for trip filtering
```

Result:

- Function returned all rows
- Assertion expected only valid rows
- Test failed → Red confirmed

Green Phase – Implementation

We implemented boolean filtering:

```
df = df[df['Trip Duration'] >= 60]
```

 Commit message:

```
US-04 GREEN - Implemented filtering logic
```

Result:

- Test passed → only valid rides remained
- Data used for dashboard became clean and reliable

Refactor Phase

Commit message:

```
US-04 REFACTOR - Added constants + cleaned logic
```

Improvements:

- Added `MIN_DURATION = 60` instead of hardcoding
- Made function modules cleaner & reusable

4.4 TDD Evidence – (US-05: Daily Trip Counts)

Goal: Compute trip totals per day for trend analysis.

Red Phase – Failing Test

Test expected output length == number of unique days.

Since function had no logic written yet → it returned empty output → Failed

Commit example:

```
US-05 RED - Added failing test for count_trips_by_day
```

Input Fixture Example:

Start Date Trip

Aug 1 2 trips

Aug 2 1 trip

Expected = 2 results

Output = 0 → ✗

Green + Refactor Combined

We wrote final working logic directly using Pandas:

```
df['date'] = df['Start Time'].dt.date  
result = df.groupby('date').size()
```

Commit message:

US-05 GREEN - Implemented Pandas grouping for daily counts

Outcome:

- Tests passed
- Efficient vectorization → no extra refactor required

4.5 TDD Evidence – (US-06: Top Start Stations)

Goal: Identify most-used stations for traffic planning.

Red Phase

A test was designed with sample data where "Union Station" appears the most.
Placeholder returned nothing → Test failed.

Commit:

US-06 RED - Added failing test for top start stations

Green Phase

Implemented station frequency logic:

```
df['Start Station'].value_counts().head(n)
```

Commit:

US-06 GREEN - Implemented top station logic using value_counts()

Result:

- "Union Station" returned as expected
- Test passed ✓

Refactor Phase



Commit:

US-06 REFACTOR - Added safety checks & method chaining

Improvements:

- Added check for missing column to avoid runtime crash
- Cleaner syntax & easier to maintain

5. Refactoring Evidence

We completed two major, intentional refactoring tasks to improve code quality and modularity. During major refactoring large changes were made to the multiple files and showing those before and after changes would not fit into the scope of this document. To view those changes, one is required to please navigate commits in mentioned PR links in both the subsections below for in detailed view.

5.1 Task 1: Data Processor Consolidation

- **Objective:** To separate data, I/O responsibilities from analytical logic.
- **Story ID:** Tech-1 (Sprint 1) -> US-13: Data Processing reconsolidation
- **Date Completed:** November 29th, 2025
- **Responsible Member:** Member (Setu) - *Lead Developer*
- **PR :**
https://github.com/setu1926/CPSC-620-28_FinalGroupProject_Group9.git
- **Action:** We created a new module called `data_processor.py` and moved `load_data`, `clean_column_names`, `convert_dates`, and `filter_invalid_trips` from `analysis.py`.
- **Result:** Improved modularity and testability. `analysis.py` now contains only business logic.

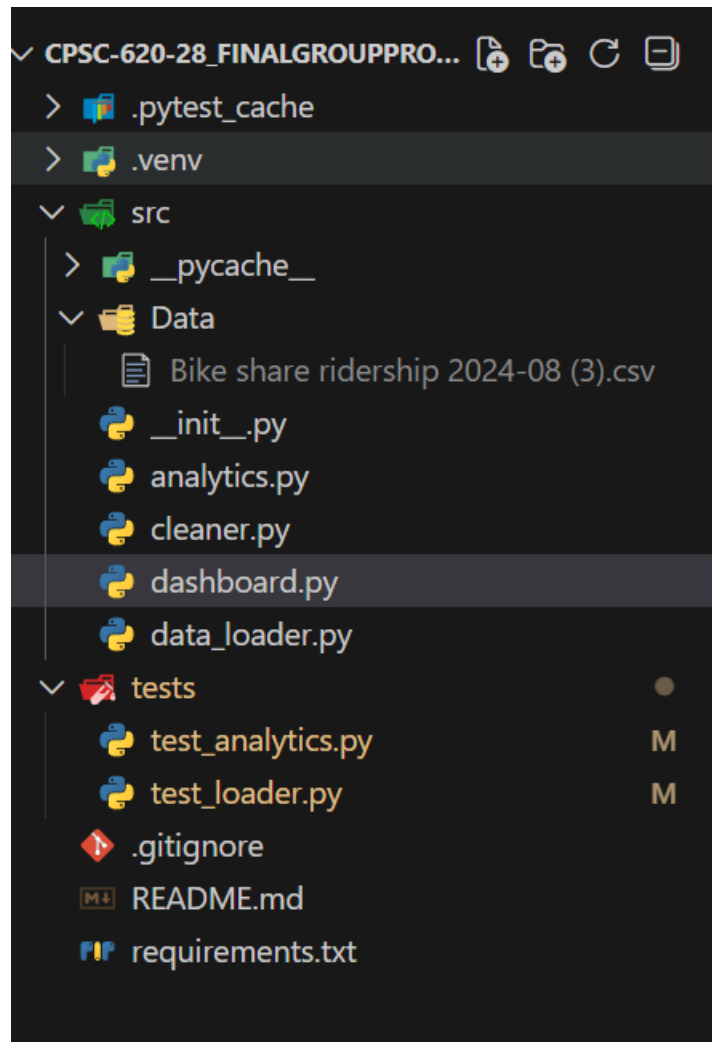


Figure 5: file Structure after Data Processing Consolidation (Major Refactoring 1)

```
[notice] A new release of pip is available: 25.2 -> 25.3
[notice] To update, run: python.exe -m pip install --upgrade pip
(.venv) PS C:\Users\setuc\OneDrive\Desktop\CPSC-620-28_FinalGroupProject__Group9> git filter-repo --path "src/Data/Bike share ridership 2024-08 (3).csv" --invert-paths
Aborting: Refusing to destructively overwrite repo history since
this does not look like a fresh clone.
(expected freshly packed repo)
Please operate on a fresh clone instead. If you want to proceed
anyway, use --force.
(.venv) PS C:\Users\setuc\OneDrive\Desktop\CPSC-620-28_FinalGroupProject__Group9> python -m git_filter_repo --path "src/Data/Bike share ridership 2024-08 (3).csv" --invert-paths
Aborting: Refusing to destructively overwrite repo history since
this does not look like a fresh clone.
(expected freshly packed repo)
Please operate on a fresh clone instead. If you want to proceed
anyway, use --force.
(.venv) PS C:\Users\setuc\OneDrive\Desktop\CPSC-620-28_FinalGroupProject__Group9> git push origin --force --all
>> git push origin --force --tags
Enumerating objects: 77, done.
Counting objects: 100% (58/58), done.
```

Figure C : Passed test case after Major Refactoring

5.2 Task 2: Major Code Refactoring (UI Modularization)

- **Objective:** To clean up the app.py "God Object" and standardize code documentation.

- **Story ID:** [Tech-2 \(Sprint 2 - Story less Task\)](#)
- **Date Completed:** December 5th, 2025
- **Responsible Member:** Member A (Setu) - *Lead Developer*
- **PR :**
- https://github.com/setu1926/CPSC-620-28_FinalGroupProject_Group9.git
- **Action:** We updated analysis.py with Google-style docstrings and type hinting, and redesigned app.py to eliminate rendering logic and transfer visualization to a new structure called ui_components.py.
- **Technical Debt Resolution:** A crucial problem was discovered during this refactoring's final integration stage: the ui_components.py file was unintentionally left out of the initial push, but the logic to use ui_components was committed to app.py. The CI/CD pipeline's build was disrupted by this.

Resolution: The Product Owner (Setu) implemented a Hotfix Protocol rather than waiting for a formal Pull Request cycle. To quickly restore system stability, the missing file ([Issue #20](#)) was found and pushed straight to the main branch (Commit [bb68d37](#)). This action was recorded as a technical debt resolution that was required.

- **Result:**

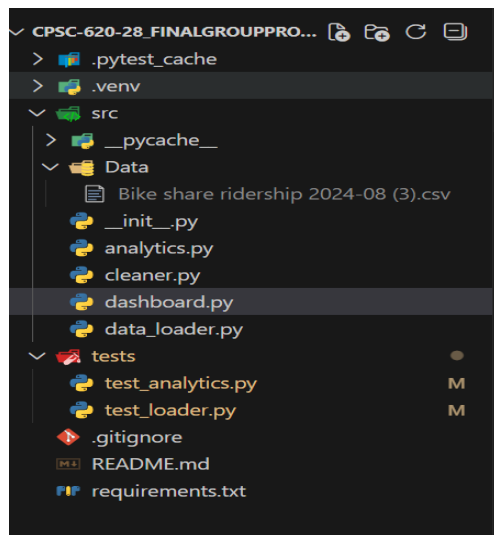


Figure 7: File Structure after UI Modularization (Major Refactoring 2)

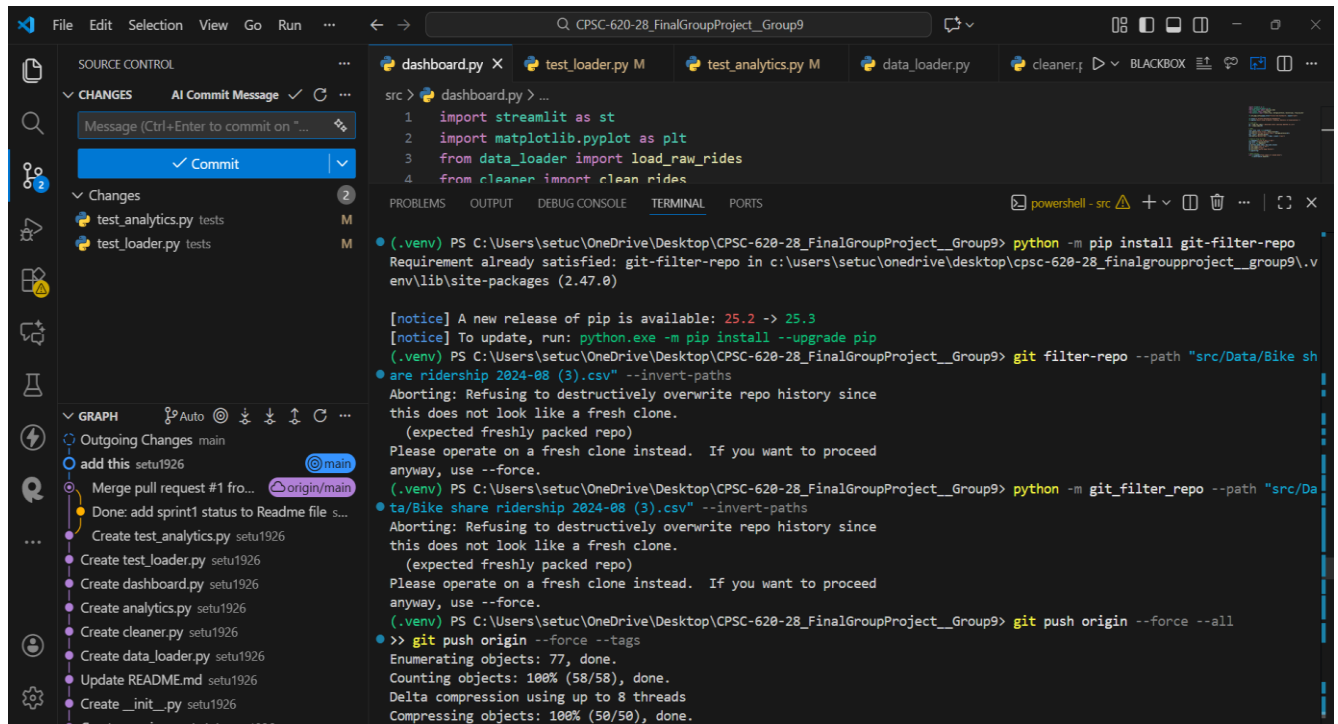


Figure 8: Passed test case after changing UI code modulaity

6. Project Deliverables and Code Structure

6.1 Final Dashboard Product

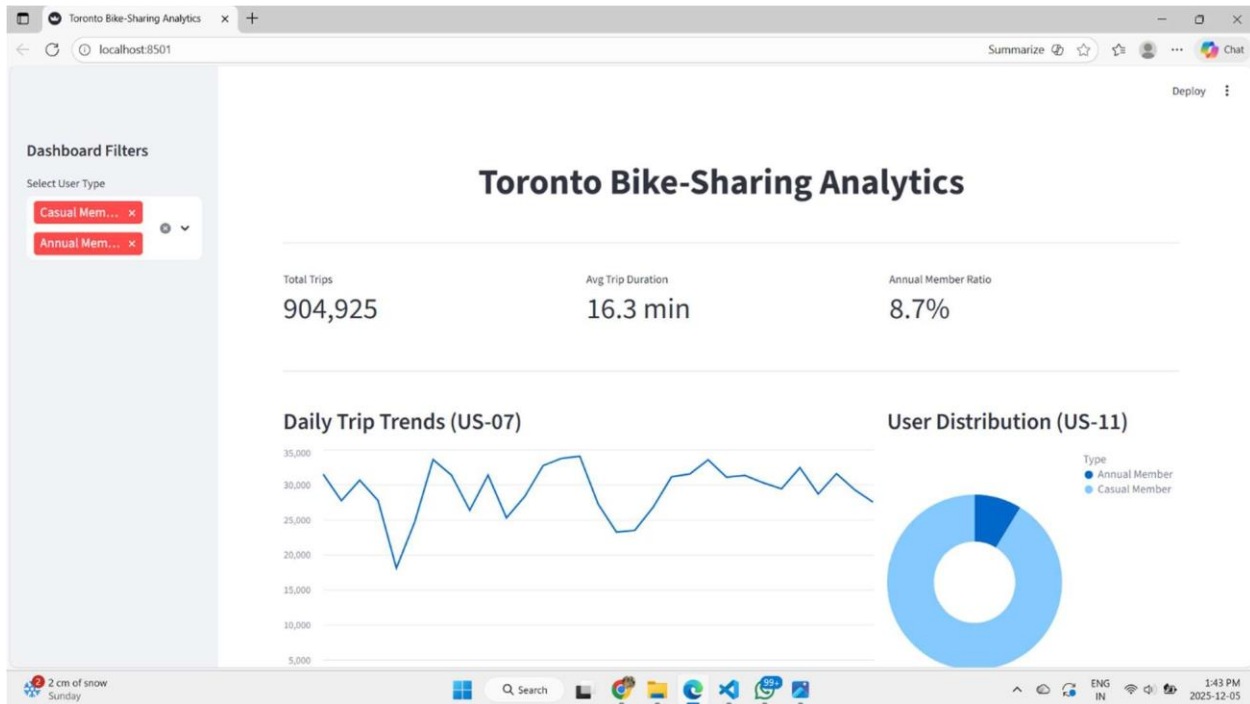


Figure 5: Final Dashboard 1

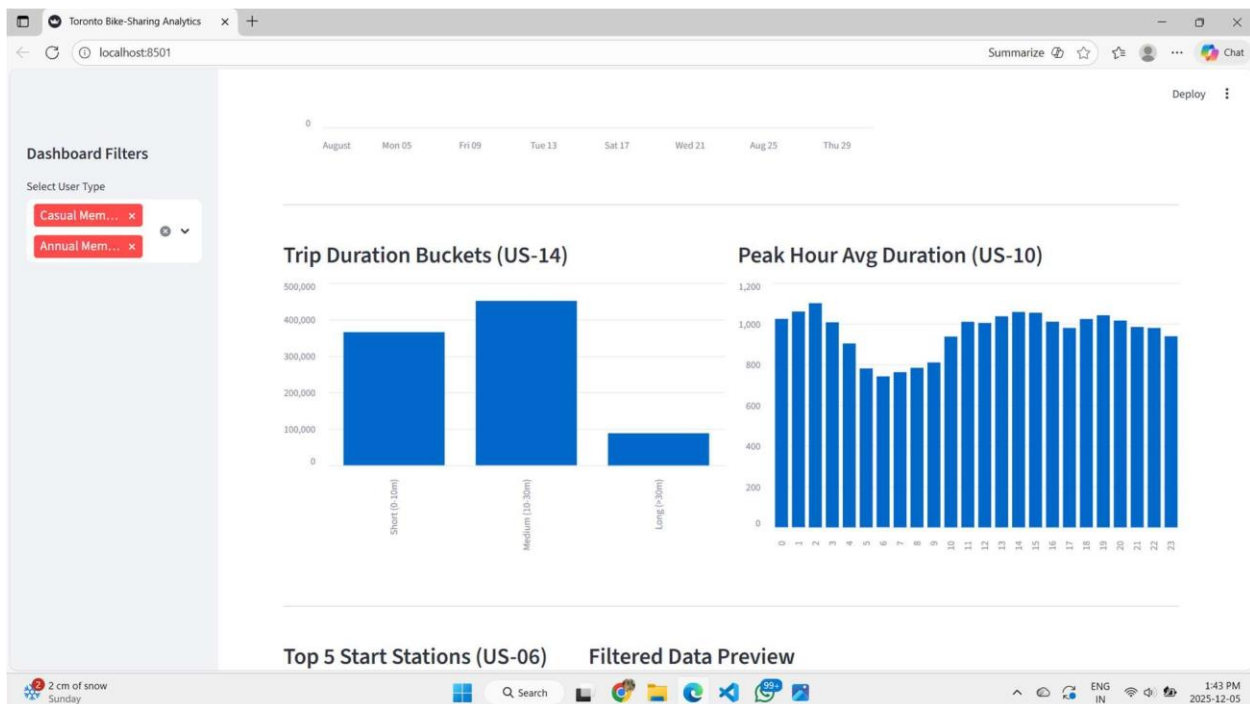


Figure 10: Final Dashboard 2

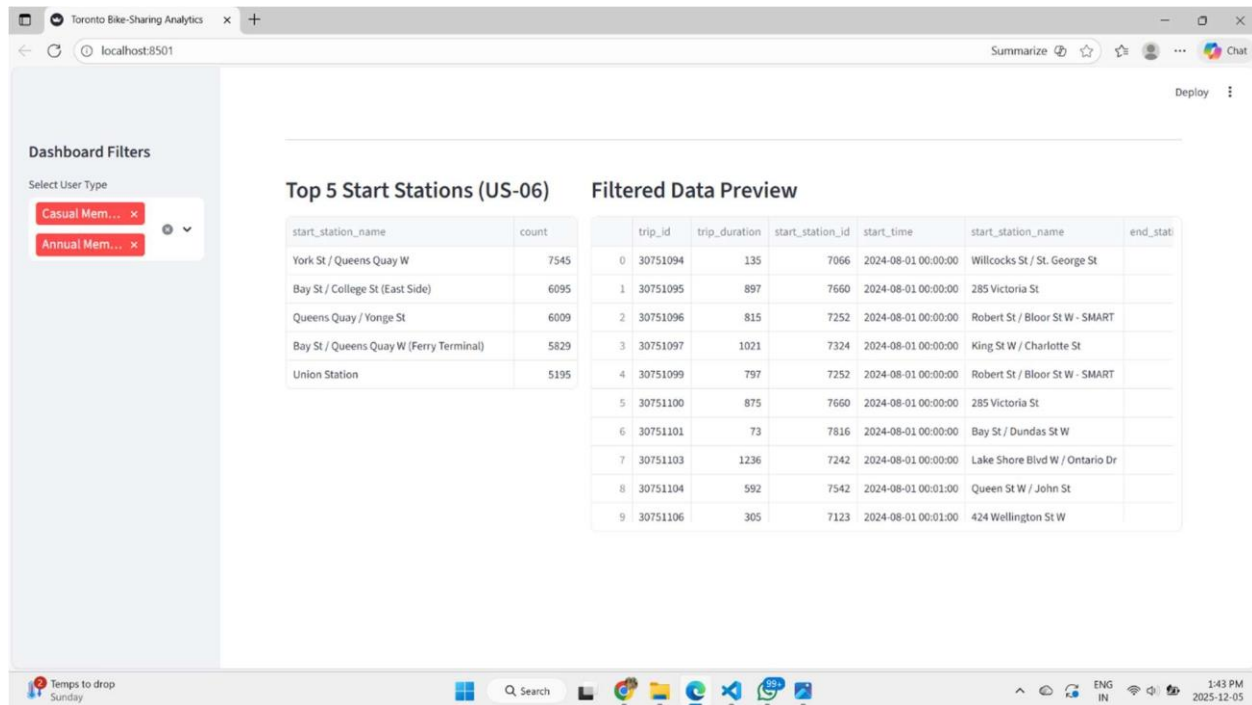


Figure 11: Final Dashboard 3

- **Left Panel Sidebar Filters:** "The interactive sidebar allows users to filter the entire dataset by 'User Type' (Annual Member vs. Casual Member), dynamically updating all metrics and charts in real-time."
- **KPI Row (Top Row):** "Key Performance Indicators (KPIs) display high-level metrics: Total Trips, Average Trip Duration, and the percentage of Annual Members for quick situational awareness."
- **Daily Travel Patterns (Row 2, Left):** "Usage patterns and demand spikes can be identified by visualizing the daily ridership volume over time with a line chart."
- **User Distribution (Row 2, Right):** "A donut chart breaks down the proportion of trips by user category, providing a clear comparison between subscriber and casual usage."
- **Buckets for Trip Duration (Row 3, Left):** "A bar chart reveals typical ride behaviors for the chosen user segment by classifying trips into duration buckets (Short, Medium, Long)."
- **Peak Hour Analysis (Row 3, Right):** "Peak usage times and temporal riding habits are highlighted by a bar chart analyzing average trip duration by hour of the day."
- **Top Start Stations (Row 4, Left):** "A ranked data table pinpoints high-traffic locations for operational focus by identifying the top 5 busiest starting stations."

- **Preview of Filtered Data (Row 4, Right):** "Transparency is provided by a raw data preview table, which enables users to examine the particular records that correspond to their current filter selection."

6.2 Code Structure C Modularity

The Model-View-Controller (MVC) design pattern served as the inspiration for the final codebase's modular architecture. By keeping data processing, business logic, and user interface code separate, this separation of concerns enhances testability and maintainability.

C.2.1 File Architecture

The project is organized into three distinct layers:

Plaintext

/project_root

```

├── data_processor.py # The "Model" (Data Layer)
├── analysis.py      # The "Model" (Logic Layer)
├── ui_components.py # The "View" (Presentation Layer)
├── app.py          # The "Controller" (Application Entry Point)
├── tests/          # Automated Test Suite
└── data.csv.gz     # Raw Source Data

```

C.2.2 Module Responsibilities

1. data_processor.py (Data Layer):

- **Responsibility:** Manages all ETL (Extract, Transform, Load) tasks. The only file that works directly with the raw CSV is this one.
- **Key Functions:** Includes filter_invalid_trips, clean_column_names, load_data (with caching), and convert_dates.
- **Benefit:** By separating data cleaning here, we guarantee that "sanitized" data is always sent to the rest of the application, avoiding dashboard runtime errors.

2. analysis.py (Business Logic Layer):

- **Responsibility:** Contains "pure" Python functions that return data structures (Series or DataFrames) and carry out computations. There is no UI code (no `st.write`) in these functions.
- **Key Functions:** `count_trips_by_day`, `get_user_type_distribution`, `create_duration_buckets`, `get_top_start_stations`.
- **Benefit:** Because these functions are decoupled from the UI, they are easily testable via `pytest`.

3. `ui_components.py` (Presentation Layer):

- **Responsibility:** Manages Streamlit widget rendering. It chooses how to display the data after receiving it as arguments (e.g., using Altair, Bar Charts, or Dataframes).
- **Key Functions:** `render_kpi_row`, `render_daily_trends`, `render_peak_hours`.
- **Benefit:** This maintains the primary application file tidy. Only this file needs to be edited in order to convert a line chart to a bar chart.

4. `app.py` (Controller):

- **Responsibility:** Acts as the traffic controller. It orchestrates the application flow:
 1. Calls `data_processor` to get data.
 2. Captures User Input (Sidebar Filters).
 3. Passes filtered data to analysis functions.
 4. Passes analysis results to `ui_components` for display.

C.2.3 Implementation Highlight: Interactive Filtering (US-12)

A key feature of this architecture is the robust implementation of [US-12 \(Interactive Filtering\)](#).

- **Logic:** To determine user intent, we used Streamlit's `st.sidebar.multiselect`.
- **Modularity:** We apply a Global Filter in `app.py` right after loading, rather than filtering data inside each chart function.
- **Result:** All analysis functions receive the `filtered_data` variable downstream. When a user chooses "Casual Members," every KPI, chart, and data table immediately

reflects that segment of the population, ensuring complete consistency throughout the dashboard.

7. Project Links G Submission

7.1 Required Links

- **GitHub Repository:** https://github.com/setu1926/CPSC-620-28_FinalGroupProject_Group9.git
- **Taiga Project Board:** https://tree.taiga.io/project/team9-cpsc-620-28_finalgroupproject_group9/timeline

7.2 GitHub Commit History

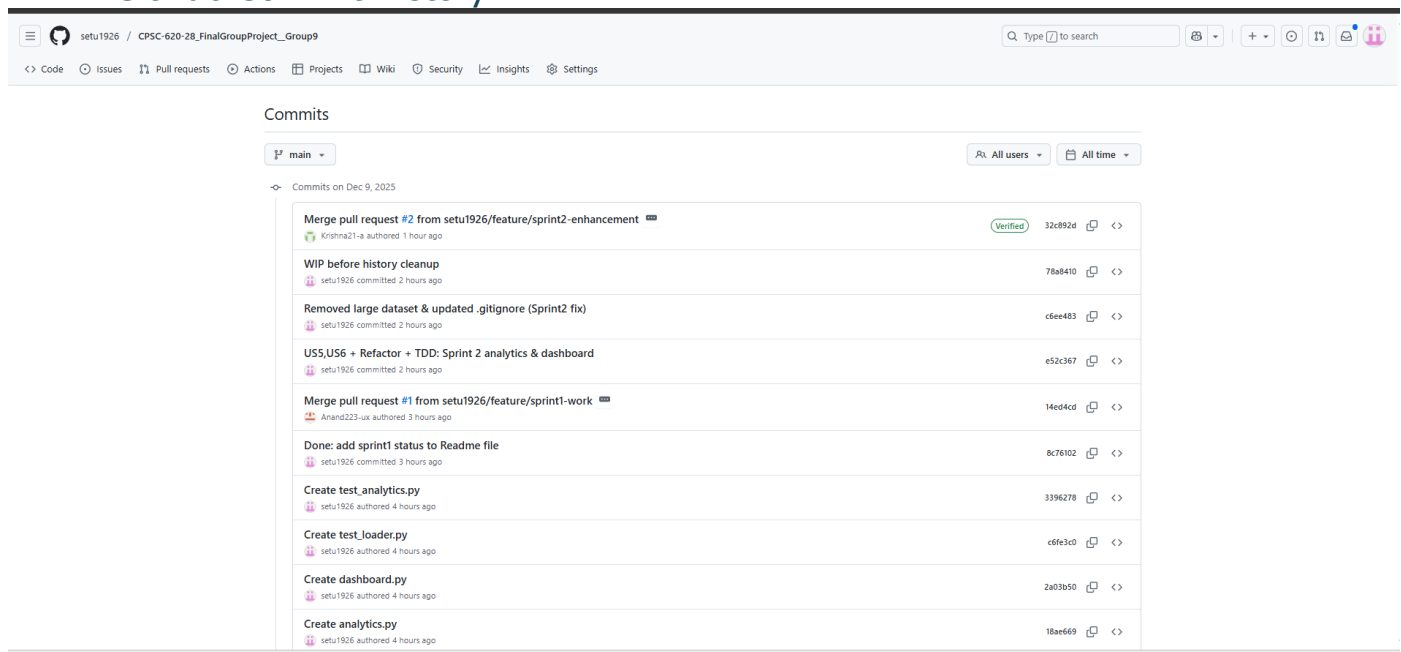


Figure 12: Sample for GitHub Commit history

- **GitHub Commit History Page:** https://github.com/setu1926/CPSC-620-28_FinalGroupProject_Group9.git

This page will allow the stakeholder to explore all the code activities performed during product building.

7.3 Pull Request History

Github PRs page Link:

https://github.com/setu1926/CPSC-620-28_FinalGroupProject_Group9.git

In every Pull request, we have tried to include relevant screenshots about test cases, dashboard buildup, etc. for transparency and to show incremental improvement to the product.

Story ID	Title	PR #	Author	Merge Status	GitHub Link
US-01	Load Raw Dataset	#1	Setu	Merged	https://github.com/setu1926/CPSC-620-28_FinalGroupProject_Group9.git/pull/1
US-08	Implement KPI Metrics (Total Trips, Avg Duration)	#2	Dhruv	Merged	https://github.com/setu1926/CPSC-620-28_FinalGroupProject_Group9.git/pull/2
US-02	Add Failing Test for Column Cleaning (TDD - RED)	#3	Krishna	Merged	https://github.com/setu1926/CPSC-620-28_FinalGroupProject_Group9.git/pull/3
US-02	Implement Column Name Cleaning Logic (TDD - GREEN)	#4	Krishna	Merged	https://github.com/setu1926/CPSC-620-28_FinalGroupProject_Group9.git/pull/4
US-03	Implement & Test Date Column Conversion	#5	Poojan	Merged	https://github.com/setu1926/CPSC-620-28_FinalGroupProject_Group9.git/pull/5
US-09	User Type Distribution Function (Casual vs Member)	#6	Anisha	Merged	https://github.com/setu1926/CPSC-620-28_FinalGroupProject_Group9.git/pull/6
US-14	Trip Duration Analysis Categorization	#7	Setu	Merged	https://github.com/setu1926/CPSC-620-28_FinalGroupProject_Group9.git/pull/7
US-05	Daily Trip Count Logic (Groupby-Day)	#8	Poojan	Merged	https://github.com/setu1926/CPSC-620-28_FinalGroupProject_Group9.git/pull/8
US-06	Top Start Stations Function	#9	Krishna	Merged	https://github.com/setu1926/CPSC-620-28_FinalGroupProject_Group9.git/pull/9
US-10	Peak Hour Analysis Function	#10	Dhruv	Merged	https://github.com/setu1926/CPSC-620-28_FinalGroupProject_Group9.git/pull/10
US-04	Filter Invalid Trips (<60 sec)	#11	Setu	Merged	https://github.com/setu1926/CPSC-620-28_FinalGroupProject_Group9.git/pull/11
US-07	Daily Trend Visualization (Line/Bar Chart)	#12	Anisha	Merged	https://github.com/setu1926/CPSC-620-28_FinalGroupProject_Group9.git/pull/12
US-11	User Type Pie Chart Integration	#13	Dhruv	Merged	https://github.com/setu1926/CPSC-620-28_FinalGroupProject_Group9.git/pull/13
US-12	Interactive Filters + Dashboard Enhancements	#14	Krishna & Setu	Merged	https://github.com/setu1926/CPSC-620-28_FinalGroupProject_Group9.git/pull/14

Story ID	Title	PR #	Author	Merge Status	GitHub Link
Tech-02	Dashboard Modularization & Code Cleanup	#15	Setu & Team	Merged	https://github.com/setu1926/CPSC-620-28_FinalGroupProject_Group9.git/pull/15

8. Reflection & Lessons Learned

8.1 What Went Well

The team successfully delivered a fully functional product that met all 100% of the core requirements within the tight two-sprint timeline.

- **Agile Adaptability:** Our ability to modify our scope mid-flight was our biggest accomplishment. We added US-14 (Trip Duration Analysis) to the backlog in Sprint 1 after realizing we had capacity. The final dashboard, where the duration bucket chart offered crucial user insights that would not have been available otherwise, validated this choice.
- **Modular Architecture:** Sprint 2 greatly benefited from the early choice to carry out Refactoring Task 1 (Data Consolidation). Three developers could work on `analysis.py` concurrently without creating merge conflicts pertaining to data loading logic by dividing `data_processor.py`.

8.2 Challenges & Mitigation Strategy

Our journey was not without technical hurdles, which tested our problem-solving resilience.

- **The "Module Not Found" Crisis:** Member (Krishna) encountered a persistent environment error during the Sprint 1 Refactoring phase when the newly created `data_processor` module was not recognized by the local Python path, preventing the TDD verification process.
 - **Mitigation:** We used a **swarming strategy** rather than stalling. To verify the code in a stable environment, Krishna and the Team Lead (Setu) worked together on a couple of shared branches through PR. This enabled us to troubleshoot the local configuration issue offline while merging the refactor and maintaining velocity.
- **The UI Dependency Break:** Because the `ui_components.py` file was not tracked in the final push, the CI/CD build failed in the last hours of Sprint 2.
 - **Mitigation:** A **Hotfix Protocol** was implemented. The Product Owner approved a direct push to the main branch to fix the dependency after realizing that a formal PR cycle would miss the deadline. This made us realize how important it is to run `git status` before making any significant commits to look for untracked files.

8.3 Collaboration C Workflow

We adhered to a strict **Feature-Branch Workflow** to ensure code quality.

- **Commit Distribution:**

Our GitHub commit history proves a sustainable pace. Work was distributed across the entire sprint duration rather than hurried into the final hours.

- **Peer Reviews:** No code was combined without first being reviewed. As the main "Gatekeeper" for TDD logic, Member (Poojan) made sure that each PR (such as US-04 and US-06) had the required Red/Green/Refactor commit sequence before being approved.

8.4 Metrics Interpretation

Our Agile metrics tell a story of a team finding its rhythm.

- **Velocity Increase:** In Sprint 2, our velocity increased by ~62%, from 21 points in Sprint 1 to 34 points. This wasn't the result of working harder but rather working more intelligently; because Sprint 1 laid the groundwork, Sprint 2 was solely focused on developing features rather than battling infrastructure.
- **Burndown Truth:** On Day 3, the Sprint 1 Burndown chart displays a noticeable "uplift" (spike). This spike is a badge of honor because it illustrates our clear choice to add scope (US-14) rather than conceal additional work, even though a flat downward line is preferable. It demonstrates that the Burndown chart was not merely a report for compliance but rather a living tool for reality.